

Системное ПО, роль системного ПО, низкоуровневое программирование

1. Что такое системное ПО и его роль

Системное ПО — это мост между прикладными программами и аппаратным обеспечением компьютера. Его главная цель — управлять ресурсами и изолировать прикладного программиста от сложностей работы с железом.

- **Классификация:** Операционные системы (ядро, драйверы), утилиты, системы управления базами данных (СУБД) и инструменты разработки (компиляторы, линкеры, отладчики).
- **Ключевые функции:** Управление памятью, планирование процессов, работа с файловыми системами и сетевыми протоколами.
- **Иллюстрация абстракции:** Покажите путь команды от прикладного уровня до транзисторов (например, вызов функции вывода текста на экран).

2. Спектр языков: от C# до NASM (Уровни абстракции)

Здесь важно показать студентам, где на шкале абстракций находятся используемые вами языки:

- **NASM (Ассемблер):** Нулевой уровень абстракции. Прямое отображение машинных команд процессора. Полный контроль над регистрами (EAX, RSP), флагами и системными вызовами (syscall).
- **C++:** Системный язык высокого уровня. Предоставляет абстракции (классы, шаблоны), но компилируется напрямую в машинный код. Дает ручное управление памятью (указатели, new/delete) и предсказуемую производительность без сборщика мусора.

- **C# (.NET):** Прикладной/платформенный уровень. Работает поверх виртуальной машины (CLR) с автоматическим управлением памятью (Garbage Collector) и JIT-компиляцией. Однако имеет механизмы спуска на низкий уровень.

3. Низкоуровневое программирование на практике

Покажите, как одна и та же задача (например, работа с памятью или вызов ядра) выглядит в трех ваших экосистемах.

Блок А: Управление памятью и указатели

- **NASM:** Работа со стеком напрямую (`push`, `pop`) и ручная адресация сегментов данных.
- **C++:** Работа с сырыми указателями (`int* p`), арифметика указателей, выделение памяти на куче (`malloc/free`) и стек-аллокация.
- **C#:** Расскажите про существование **указателей в C#**. Покажите ключевое слово `unsafe`, `fixed`-указатели (`fixed (int* p = ...)`), структуры `Span<T>` и `ValueTask` для оптимизации выделения памяти без участия GC.

Блок Б: Взаимодействие между языками (Interoperability)

Расскажите, как системный код связывается воедино:

- Как вызвать функцию, написанную на NASM, внутри C++ проекта (соглашения о вызовах: `__cdecl`, `__fastcall`, `__stdcall`).
- Как из C# вызвать низкоуровневую Win32 API функцию или C++ библиотеку через **P/Invoke** (`[DllImport]`).

4. Роль компилятора и линкера (Сборка кода)

Студенты должны понимать, во что превращается их код:

1. **Трансляция:** Как C++ и NASM превращаются в объектные файлы (`.obj` / `.o`).

2. **Линковка:** Зачем нужен компоновщик, как разрешаются внешние ссылки, в чем разница между статической (.lib / .a) и динамической (.dll / .so) линковкой.
3. **Платформозависимость:** Почему скомпилированный код C++ или NASM привязан к конкретной архитектуре процессора (x86/x64) и ОС, а C# (IL-код) — переносим.

5. Живые примеры для демонстрации (Live Coding)

Чтобы лекция не была скучной, покажите пару контрастных примеров:

- **Пример 1: Вывод "Hello World" на NASM против C#**
Покажите, сколько магии скрывает C# за одной строчкой `Console.WriteLine`, продемонстрировав на ассемблере прямую запись в регистры и вызов прерывания/системного вызова (`sys_write`).
- **Пример 2: Инлайн-ассемблер или оптимизация в C++**
Покажите, как компилятор C++ оптимизирует циклы (SIMD-инструкции, векторизация), и как этот код выглядит в отладчике на уровне инструкций ассемблера.

Вот подробный разбор первого раздела лекции «Что такое системное ПО и его роль». Этот материал рассчитан примерно на 15–20 минут изложения и снабжен понятными метафорами, которые помогут студентам усвоить базу.

Раздел 1. Системное ПО: Определение, роль и архитектурные границы

1.1. Проблема, которую решает системное ПО (Зачем оно нужно?)

Начните с контрпримера: представьте, что системного ПО не существует.

- Если прикладной программист хочет просто сохранить текстовый файл на диск, ему придется вручную подавать команды контроллеру жесткого диска или SSD.

- Ему нужно знать, на каком физическом секторе диска лежат данные, как управлять магнитной головкой или ячейками памяти, и как не затереть данные других программ.
- **Вывод:** Без системного ПО написание даже простейшего калькулятора превратилось бы в инженерный кошмар. Системное ПО берет рутину управления железом на себя.

1.2. Определение и ключевая роль

Системное ПО — это комплекс программ, которые управляют аппаратными ресурсами компьютера и обеспечивают среду для запуска и выполнения прикладного софта.

Его главная роль — создание абстракций. Оно превращает сложное, нестабильное и разнородное «железо» (сотни видов видеокарт, процессоров, накопителей) в понятный, предсказуемый и унифицированный набор функций для прикладного разработчика.

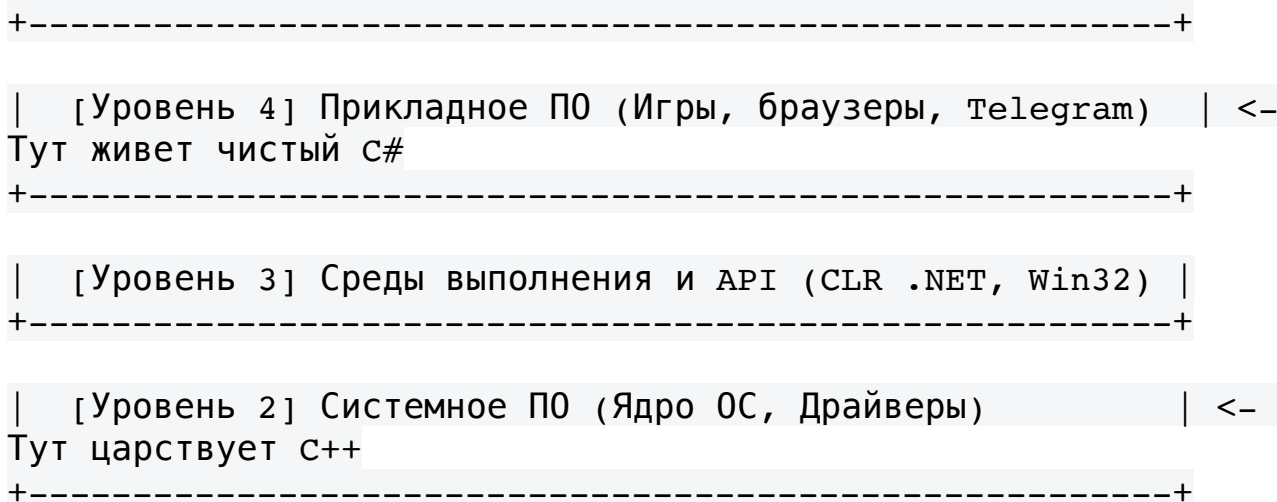
1.3. Классификация системного ПО (Из чего оно состоит)

Чтобы у студентов сложилась четкая картина, разделите системное ПО на четыре основные группы:

1. **Операционные системы (ОС)** — фундамент. Именно они распределяют процессорное время и оперативную память между программами.
2. **Драйверы устройств** — «переводчики». Это специальные модули, которые переводят стандартные запросы ОС на специфический язык конкретного «железа» (например, видеокарты Nvidia или сетевой платы Realtek).
3. **Утилиты и сервисы** — обслуживающие программы. Сюда относятся дефрагментаторы, диспетчеры задач, архиваторы, средства резервного копирования и системные мониторы.
4. **Инструменты разработки (Инструментальное ПО)** — компиляторы (для C++/C#), ассемблеры (NASM), линкеры и отладчики. Они также считаются системными, так как работают напрямую с генерацией машинного кода и анализом памяти.

1.4. Архитектурная визуализация: «Слоеный пирог» ЭВМ

Нарисуйте на доске или покажите на слайде классическую схему уровней абстракции. Это визуально свяжет тему с вашим стеком (C#, C++, NASM):



| [Уровень 1] Аппаратное обеспечение (CPU, RAM, GPU) | <—
Сюда бьет NASM (Ассемблер)

+-----+

Объяснение схемы: Каждый уровень доверяет уровню ниже и ничего не знает о деталях его реализации. Прикладная программа не знает, как устроен процессор; она просит ОС выделить память.

1.5. Демонстрация абстракции: Путь одной команды

Покажите студентам, что происходит «под капотом», когда прикладная программа (например, на C#) выполняет команду записи строки в файл:

1. **C# (File.WriteAllText):** Программа на C# обращается к стандартной библиотеке .NET. .NET проверяет безопасность и передает запрос ниже.
2. **Подсистема ОС (API):** .NET вызывает функцию операционной системы (например, CreateFile/WriteFile в Windows API, написанные на C/C++).
3. **Файловая система:** Ядро ОС проверяет права доступа пользователя и определяет, в какое именно место файловой системы (NTFS/ext4) нужно записать данные.
4. **Драйвер диска:** ОС передает команду драйверу накопителя (SSD). Драйвер переводит это в низкоуровневый протокол (например, NVMe или SATA).
5. **Железо:** Контроллер SSD подает электрические импульсы на флеш-память, физически сохраняя биты информации.

Главный вывод этого пункта для лекции: Системное ПО — это невидимый диспетчер. Чем лучше оно работает, тем меньше прикладной программист замечает его существование.

Если вы хотите развить эту часть, я могу детально расписать **разницу между пространством пользователя (User Space) и пространством ядра (Kernel Space)**, что очень важно для понимания системных вызовов в C++ и NASM. Рассказать об этом?

Переходим к детальному разбору **Пункта 2: «Спектр языков: от C# до NASM (Уровни абстракции)»**.

Этот раздел лекции критически важен. Его цель — объяснить студентам, почему в мире до сих пор сосуществуют такие разные языки, почему нельзя всё написать на «удобном» C# или «быстром» NASM, и как они соотносятся с аппаратной частью компьютера.

Раздел 2. Спектр языков: от C# до NASM (Уровни абстракции)

Главная идея раздела: **«Чем выше уровень абстракции, тем ближе язык к человеку и дальше от железа. Чем ниже уровень — тем ближе язык к процессору, но сложнее для человека»**.

Высокий уровень (Человек)
<-----> Низкий уровень
(Железо)

[C#]	[C++]
[NASM]	
Виртуальная машина (CLR)	Родной код ОС (Native)
Инструкции процессора	
Управляет памятью сама	Управляешь памятью сам
Управляешь регистрами	

2.1. NASM (Ассемблер): Нулевой уровень абстракции

Ассемблер — это текстовая человекочитаемая форма **машинного кода**. Здесь нет понятий «переменная», «класс», «цикл for» или «функция».

- **С чем работает код:** Напрямую с архитектурой процессора. Основные сущности — это регистры процессора (**RAX, RBX, RSP**), оперативная память по конкретным адресам и стек.
- **Как это работает:** Каждая строчка кода на NASM транслируется почти ровно в одну машинную команду (инструкцию процессора), например:
 - `mov rax, 1` — записать число 1 в регистр RAX.
 - `add rax, rbx` — сложить значения в регистрах RAX и RBX.
 - `syscall` — передать управление ядру операционной системы.
- **Плюсы:** Абсолютный контроль над процессором. Можно выжать 100% скорости из железа и написать программу размером в несколько килобайт.
- **Минусы:** Код, написанный для процессора Intel (x86/x64), не запустится на процессоре смартфона (ARM). Ошибиться невероятно легко: забыли очистить стек — программа намертво упала.

2.2. C++: ни

C++ — это уникальный гибрид. С одной стороны, он дает абстракции высокого уровня (классы, шаблоны, ООП). С другой — он компилируется прямо в машинный код для конкретной ОС и процессора, не требуя никаких «посредников» при запуске.

- **С чем работает код:** С типизированными переменными, объектами и **указателями**. Указатель (`int* p`) — это и есть прямой адрес в оперативной памяти компьютера.
- **Философия C++:** «*Вы платите только за то, что используете*». В C++ нет автоматического сборщика мусора. Если программист выделил кусок памяти командой `new`, он обязан сам вернуть её системе командой `delete`.
- **Почему он системный:** На C++ пишутся ядра операционных систем (Windows, Linux, macOS), драйверы, игровые движки и СУБД. Он предсказуем: код выполнится ровно тогда и так, как его написал программист, без внезапных пауз на очистку памяти.

2.3. C# (.NET): Платформенный/Прикладной уровень

C# создан для того, чтобы программист думал о бизнес-логике приложения, а не о байтах и регистрах. Он работает на базе платформы .NET и виртуальной машины CLR (**Common Language Runtime**).

- **Как это работает:** Код на C# компилируется не в машинный код, а в промежуточный байт-код (IL — **Intermediate Language**). Когда пользователь запускает программу, CLR «на лету» (это называется **JIT-компиляция** — Just-In-Time) превращает этот IL-код в машинные инструкции под текущий процессор.
- **Главный защитник — Garbage Collector (GC):** Программист в C# просто создает объекты (`new MyClass()`), а виртуальная машина сама следит, когда они станут ненужными, и автоматически очищает память.
- **Плюсы:** Кроссплатформенность (один бинарник работает на Windows, Linux и Mac через .NET Core), безопасность (нельзя случайно затереть чужую память) и огромная скорость разработки.
- **Минусы:** Потребляет больше оперативной памяти. Сборщик мусора может внезапно «заморозить» программу на доли секунды для уборки, что недопустимо в реальном времени (например, в системах управления автомобилем или роботами).

Метафора для студентов (Как это объяснить на пальцах)

Представьте, что вам нужно построить дом (написать программу):

1. **NASM** — это когда вы сами вручную обжигаете каждый кирпич из глины и замешиваете цемент. Это долго, тяжело, но если вам нужен кирпич уникальной формы — это единственный путь.
2. **C++** — это когда вам привезли готовые кирпичи и бетон, но класть их и строить несущие стены вы должны строго сами. Ошибетесь в расчетах — дом (программа) рухнет.
3. **C#** — это строительство из готовых крупных 3D-модулей на заводе. Вы просто собираете блоки вместе. Это быстро, безопасно, дом точно будет стоять, но изменить внутреннюю структуру самого кирпича вы уже не можете.

Переходим к **Пункту 3 («Низкоуровневое программирование на практике» с примерами кода)** или вы хотите что-то уточнить по уровням абстракции?

Переходим к **Пункту 3: «Низкоуровневое программирование на практике».**

Здесь лекция становится максимально прикладной. Задача — показать студентам, что низкий уровень — это не просто абстрактная теория, а конкретные приемы, которые связывают NASM, C++ и C#.

Разберите эту тему через два больших блока: **управление памятью и взаимодействие языков.**

Раздел 3. Низкоуровневое программирование на практике

Блок А: Управление памятью и указатели

Память процесса делится на две ключевые зоны: **Стек (Stack)** — быстрый, линейный, управляется автоматически; и **Куча (Heap)** — динамическая, большая, но требует накладных расходов.

1. NASM (Архитектура x86-64)

В ассемблере нет переменных в привычном смысле. Есть регистры процессора и адреса в оперативной памяти.

- **Стек:** Управляется вручную через регистр `RSP` (указатель стека). Команды `push` и `pop` физически двигают этот указатель вверх и вниз. Разработчик обязан следить, чтобы баланс стека не нарушился к моменту выхода из функции (`ret`).
- **Доступ к памяти:** Происходит через разыменование регистров. Например, `mov rax, [rbx]` означает: «Посмотри, какой адрес лежит в `rbx`, пойдя по этому адресу в RAM, возьми оттуда данные и положи в `rax`». Это чистейшая работа с адресами.

2. C++ (Native-код)

C++ дает полный контроль над обоими типами памяти и вводит понятие **указателя** (переменной, хранящей адрес памяти).

```
// Выделение памяти на стеке (быстро, удалится само при выходе из функции)
```

```
int stackArray[100];
```

```
// Выделение памяти на куче (вручную, нужно удалять самому)
```

```
int* heapArray = new int[100];
```

```
// Арифметика указателей (сдвиг по адресу памяти)
```

```
int* secondElement = heapArray + 1; // Сдвиг на sizeof(int) байт вперед
```

```
delete[] heapArray; // Если забыть — будет утечка памяти (Memory Leak)
```

3. C# (Managed, но со спуском на низкий уровень)

Обычно C# полностью изолирует программиста от памяти с помощью Garbage Collector (GC). Но если нужна пиковая производительность, C# позволяет спуститься на уровень C++.

- **Ключевое слово `unsafe`:** Разрешает использовать сырые указатели, как в C++.
- **Конструкция `fixed`:** Запрещает сборщику мусора перемещать объект в памяти, пока мы работаем с его адресом. GC любит двигать объекты для оптимизации кучи, и если он сдвинет объект, наш указатель станет «битым». `fixed` временно «пришпиливает» объект к месту.

```
// Для работы этого кода в свойствах проекта должен быть включен флаг "Allow unsafe code"
```



```
unsafe
{
    int[] numbers = { 10, 20, 30 };

    // Фиксируем массив в памяти, чтобы GC его не переместил
    fixed (int* p = numbers)
    {
        int* second = p + 1; // Арифметика указателей в C#
        Console.WriteLine(*second); // Выведет 20
    }
}
```

- *Примечание для лектора:* Упомяните современные структуры **Span<T>** и **Memory<T>** в C#. Они позволяют безопасно и без выделения лишней памяти в куче работать со срезами массивов, строк или нативной памяти.

Блок Б: Взаимодействие между языками (Interoperability)

Системный код редко пишется на одном языке. Обычно ядро или критичный по скорости модуль пишут на NASM/C++, а интерфейс или логику — на C#. Как заставить их работать вместе?

1. Связка C++ и NASM (Соглашения о вызовах)

Чтобы вызвать функцию из ассемблера в C++, они должны договориться, как передавать аргументы через регистры и стек. Это называется **Calling Convention** (`__cdecl`, `__fastcall`, `__stdcall`). В x64 Windows стандарт один — первые 4 аргумента передаются через регистры RCX, RDX, R8, R9, остальные через стек.

Код на NASM (file.asm):

```
global asm_add ; Делаем функцию видимой снаружи
```

```
section .text
asm_add:
    ; В x64 Windows: первый аргумент в RCX, второй в RDX
    mov rax, rcx
    add rax, rdx
    ret ; Результат возвращается в регистре RAX
```

Код на C++ (main.cpp):

```
extern "C" long long asm_add(long long a, long long b); //
Объявляем внешнюю функцию
```

```
int main() {
    long long result = asm_add(5, 7); // Вызов ассемблерного
    // кода из C++
    return 0;}
}
```

2. Связка C# и C++ / Win32 API (P/Invoke)

Платформа .NET умеет вызывать функции из скомпилированных динамических библиотек (.dll в Windows или .so в Linux) с помощью механизма **Platform Invoke**.

Представим, что мы хотим вызвать функцию `MessageBox` напрямую из системной библиотеки Windows `user32.dll`:

```
using System;
using System.Runtime.InteropServices; // Необходим для работы
с нативным кодом

class Program
{
    // Импортируем функцию из системной DLL
    [DllImport("user32.dll", CharSet = CharSet.Unicode)]
    public static extern int MessageBox(IntPtr hWnd, string
text, string caption, uint type);

    static void Main()
    {
        // Вызываем низкоуровневое Win32 API окно из
управляемого C# кода
        MessageBox(IntPtr.Zero, "Привет из низкого уровня!",
"Системное ПО", 0);
    }
}
```

Главный вывод раздела для студентов

Низкоуровневое программирование — это не обязательно написание всей программы на ассемблере. Современный подход — писать 90% логики на удобном C#, а критические по скорости или работе с железом участки выносить в C++ или NASM и связывать их через P/Invoke и нативные вызовы.

Переходим к **Пункту 4: «Роль компилятора и линкера (Сборка кода)»**? Вы узнаете, как весь этот разношерстный код превращается в один исполняемый файл.

Переходим к **Пункту 4: «Роль компилятора и линкера (Сборка кода)»**.

Цель этого раздела — убрать из голов студентов опасное заблуждение, что программирование заканчивается на нажатии кнопки «Run» в IDE. Они должны четко понимать, во что превращается их код и как разрозненные файлы исходного текста собираются операционной системой в единый работающий процесс [Quiz].

Раздел 4. Роль компилятора и линкера (Как рождаются программы)

Когда мы пишем код на C++, NASM или C#, процессор не может прочитать его напрямую. Компьютер понимает только бинарный машинный код (нули и единицы). Процесс превращения текста в исполняемый файл состоит из нескольких критически важных этапов.

4.1. Путь C++ и NASM: Нативная компиляция (Компилятор -> Линкер)

Для C++ и ассемблера процесс сборки жестко привязан к целевой платформе (ОС и архитектуре процессора) и делится на две главные фазы:

Этап 1: Компиляция (Превращение текста в объектный код)

- **Что происходит:** Компилятор C++ (например, MSVC, GCC) или ассемблер (NASM) берет каждый файл исходного кода (`.cpp` или `.asm`) по отдельности и переводит его в машинные инструкции.
- **Результат:** На выходе получаются **объектные файлы** (`.obj` в Windows, `.o` в Linux).
- **Важная деталь:** Объектный файл — это уже машинный код, но его еще *нельзя* запустить. В нем вместо конкретных адресов памяти и вызовов внешних функций (например, функций вывода на экран) стоят временные «заглушки» и таблицы символов. Файлы пока ничего не знают друг о друге.

Этап 2: Линковка / Компоновка (Сборка мозаики)

- **Что происходит:** Специальная программа — **линкер (компоновщик)** — берет все ваши объектные файлы, а также внешние библиотеки, и сшивает их в один монолит.
- **Главная задача линкера:** Разрешить внешние ссылки. Если в файле `main.cpp` вызывается функция `asm_add` из файла `file.asm`, линкер находит точное местоположение кода `asm_add` и заменяет «заглушку» на реальный адрес перехода в памяти.
- **Результат:** Готовый исполняемый файл (`.exe` или ELF-бинарник без расширения в Linux).

4.2. Статическая и динамическая линковка (В чем разница?)

Студенты часто путаются в форматах библиотек. Объясните им разницу на пальцах:

- **Статическая линковка (`.lib` / `.a`):** Линкер физически копирует код функции из библиотеки и «вшивает» его прямо внутрь вашего `.exe` файла.
 - *Плюс:* Программа автономна, ей не нужны сторонние файлы для запуска.
 - *Минус:* Огромный размер файла. Если 10 программ используют одну и ту же библиотеку, она будет продублирована на диске 10 раз.
- **Динамическая линковка (`.dll` / `.so`):** Линкер не копирует код. Он просто оставляет в `.exe` файле пометку: «При запуске программы найди на компьютере файл `library.dll` и возьми функцию оттуда».

- *Плюс:* Экономия памяти. Одна DLL-ка в оперативной памяти может обслуживать десятки запущенных программ одновременно.
- *Минус:* Проблема «Ад DLL» (DLL Hell). Если удалить, переименовать или заменить файл `.dll` на другую версию, программа выдаст ошибку и откажется запускаться.

•

4.3. Путь C#: Платформенная компиляция (Двухэтапная сборка)

У C# процесс принципиально отличается ради обеспечения безопасности и кроссплатформенности.

1. **Компиляция в IL (Время сборки):** Компилятор `csc` берет код на C# и переводит его не в машинный код конкретного процессора, а в промежуточный байт-код — **IL (Intermediate Language)**. На выходе получается `.dll` или `.exe` файл, но внутри него нет инструкций для процессора Intel или AMD. Там лежат абстрактные команды для виртуальной машины .NET.
2. **JIT-компиляция (Время запуска):** Когда пользователь запускает C#-программу, управление перехватывает **CLR (Common Language Runtime)**. Специальный встроенный компилятор **JIT (Just-In-Time)** берет IL-код и прямо «на лету», прямо в оперативной памяти, превращает его в настоящие машинные команды под тот процессор, на котором программа выполняется прямо сейчас.

Сводная шпаргалка для лекции: Сравнение подходов

Критерий	C++ / NASM	C# (.NET)
Что на выходе сборки	Нативный машинный код под конкретную ОС/архитектуру	Промежуточный байт-код (IL)
Переносимость	Нет. Скомпилированный под Windows x64 код не запустится на Linux или ARM	Да. Один и тот же IL-код запустится везде, где установлена среда .NET
Когда происходит оптимизация	Во время компиляции проекта (флаги <code>-O2</code> , <code>-O3</code>).	Частично при сборке, частично прямо во время работы (JIT учитывает особенности текущего CPU).

Переходим к финальному **Пункту 5: «Живые примеры для демонстрации (Live Coding)»?** Покажу, как эффектно сравнить ассемблерный вызов и «магию» C# на экране.

Переходим к финальному **Пункту 5: «Живые примеры для демонстрации (Live Coding)».**

Цель этого раздела на лекции — перевести дух после сложной теории и показать студентам наглядные, контрастные примеры. Ничто так не объясняет суть системного ПО и уровней абстракции, как сравнение кода, выполняющего одну и ту же задачу на разных этажах этой системы.

Раздел 5. Живые примеры для демонстрации (Live Coding)

Пример 1. Великая иллюзия «Hello, World!»

Покажите студентам, сколько скрытой работы выполняет системное ПО ради одной строки текста на экране.

Вариант на C# (Высокий уровень)

```
System.Console.WriteLine("Hello, World!");
```

Что сказать студентам: «Для вас это одна понятная строчка. Но под капотом виртуальная машина .NET должна найти кодировку строки, выделить под нее память, проверить безопасность, вызвать Win32 API функцию ядра, которая передаст строку драйверу терминала».

Вариант на NASM под Windows x64 (Низкий уровень)

А вот как этот процесс выглядит, если убрать посредников и общаться с операционной системой напрямую через Win32 API.

```
bits 64
```

```
default rel
```

```
; Импортируем функции из системной библиотеки windows  
(kernel32.dll)
```

```
extern GetStdHandle
```

```
extern WriteFile
```

```
extern ExitProcess
```

```
section .data
```

```
    msg db "Hello, World!", 13, 10 ; Наша строка с переносом  
строки (CRLF)
```

```
    msg_len equ $ - msg           ; Вычисляем длину строки  
динамически
```

```
section .bss
```

```
    bytes_written resq 1           ; Выделяем 8 байт под  
переменную для результата
```

```
section .text
```

```

global main
main:
    ; 1. Получаем хендл (указатель) на стандартный вывод
    (консоль)
    ; Функция GetStdHandle принимает один аргумент. В x64 это
    регистр RCX.
    ; -11 означает STD_OUTPUT_HANDLE
    mov rcx, -11
    sub rsp, 40 ; Выделяем теневой стек
    (Shadow Space), требование Windows x64
    call GetStdHandle
    add rsp, 40 ; Возвращаем стек обратно
    mov rbx, rcx ; Сохраняем полученный
    хендл консоли в регистр RBX

    ; 2. Вызываем WriteFile(handle, buffer, length,
    &bytes_written, reserved)
    ; Аргументы передаем строго по соглашению вызовов Windows
    x64: RCX, RDX, R8, R9 + стек
    sub rsp, 40 ; Выделяем теневой стек
    mov rcx, rbx ; 1-й аргумент: хендл
    консоли
    lea rdx, [msg] ; 2-й аргумент: указатель
    на нашу строку
    mov r8, msg_len ; 3-й аргумент: длина
    строки
    lea r9, [bytes_written] ; 4-й аргумент: куда
    записать количество выведенных байт
    mov qword [rsp + 32], 0 ; 5-й аргумент:
    передается через стек (пропускаем теньевую зону в 32 байта)
    call WriteFile
    add rsp, 40 ; Восстанавливаем стек

    ; 3. Завершаем процесс: ExitProcess(0)
    mov rcx, 0 ; Аргумент 0 — код
    успешного завершения
    call ExitProcess

```

Главный вывод примера: Ассемблерный код обнажает анатомию операционной системы. Студенты увидят, что такое регистры, зачем нужен теневой стек (Shadow Space) в Windows и как работает жесткое соглашение о вызовах (Calling Convention).

Пример 2. Заглядываем под капот C++ (Разница между Debug и Release)

Этот пример отлично демонстрирует **роль компилятора как системного инструмента**, который может переписать код программиста до неузнаваемости ради оптимизации под железо.

Напишите простую функцию на C++:

```
int square_sum(int n) {  
    int sum = 0;  
    for (int i = 0; i < n; ++i) {  
        sum += i * i;  
    }  
    return sum;  
}
```

Откройте прямо на лекции Compiler Explorer (godbolt.org) или переключитесь в Visual Studio в режим отладки «Disassembly» (Отладка -> Окна -> Ассемблерный код). Поочередно скомпилируйте код:

1. В режиме Debug (Без оптимизаций, -O0):

Покажите студентам, что компилятор сгенерировал громоздкий код. Там будет честный цикл, постоянные прыжки по памяти (`mov`, `add`, `jmp`, `cmp`), чтение и запись в стек на каждой итерации. Код в точности повторяет то, что написано в текстовом файле.

2. В режиме Release (С максимальной оптимизацией, -O2 или -O3):

Покажите магию. Современные компиляторы (MSVC, GCC, Clang) увидят математическую закономерность. Вместо медленного пошагового цикла они могут применить автовекторизацию (SIMD-инструкции процессора, вроде `paddq` или `movdqu`), чтобы считать по 4 или 8 итераций за один такт процессора. А в некоторых случаях компилятор и вовсе заменит весь цикл на готовую математическую формулу суммы квадратов.

Главный вывод примера: Компилятор — это не просто переводчик, это глубокий аналитик. Низкоуровневое программирование на C++ заключается в том числе в умении писать код так, чтобы компилятор мог эффективно превратить его в максимально быстрые инструкции для процессора.

Заключение лекции (Итог для студентов)

Завершите лекцию мыслью о том, что хороший инженер не заперт на одном уровне абстракции. Понимая системное ПО и низкий уровень (NASM/C++), вы будете писать куда более эффективный, осознанный и быстрый код на высоком уровне (C#), зная, как именно ваши объекты и потоки распределяются операционной системой по реальным ячейкам памяти и ядрам процессора.

Эти языки позволяют более эффективно управлять ресурсами системы и оптимизировать производительность программ. Они используются для создания приложений, которые требуют максимальной производительности и точного контроля аппаратных ресурсов. Такие языки программирования обладают

меньшим уровнем абстракции, чем высокоуровневые языки, и предоставляют разработчикам более прямой контроль над аппаратурой.

Низкоуровневое программирование применяется во многих областях, где требуется жесткий контроль над устройством, таких как военная и космическая сферы, медицинские приложения, инженерия и др. Несмотря на сложность использования низкоуровневых языков, они являются базовыми для построения компьютерных технологий и присутствуют в каждом компьютере.

В начале компьютерной эры программирование производилось на машинном коде, который состоял из комбинации двух цифр: “1” и “0”. Это был самый примитивный язык, который обеспечивал контроль над железом компьютера на самом низком уровне. С развитием технологий и увеличением сложности ПО, написание кода на двоичном языке стало невозможным. Поэтому были созданы, более удобные для разработки ПО, низкоуровневые языки программирования, которые все еще позволяли разработчикам управлять аппаратным обеспечением компьютера.

Низкоуровневое программирование предполагает более прямой доступ к аппаратуре компьютера, что требует от специалиста более высокой квалификации. К таким языкам относятся, например, Assembler, C и C++. К низкоуровневым языкам программирования также относят: Rust, Ada, Fortran и Pascal.

Некоторые из этих языков были разработаны во времена, когда компьютеры были менее мощными и требовали более низкоуровневого программирования. Несмотря на это, они все еще используются сегодня в различных сферах, таких как автомобильная промышленность, микроконтроллеры и промышленное управление.